```
permit(
    principal in Group::"admin",
    action == Action::"delete",
    resource
);
```

This policy doesn’t just express the access logic; it explicitly limits the scope of the policy to apply only to admins performing the delete action, regardless of what the condition states. Rego doesn’t support this kind of structured scoping, so the corresponding policy would be written like this:

```
package acme.authz

import rego.v1

default allow := false

allow if {
    input.action == "delete"
    input.principal.groups[_] == "admin"
}
```

While the logic is similar, the way policy intent is expressed differs significantly. In Rego, all constraints—on the principal, action, and resource—must be written as conditions within the rule body. This provides the policy author flexibility but also puts the burden on them to manually manage scope and semantics. You cannot declare, for example, that a rule applies only to specific types of users or actions; you can only require certain conditions to be satisfied.

Cedar, in contrast, enables constraints to be declared structurally within the policy head. This enhances clarity, minimizes the risk of unintended application, and allows for performance optimizations. Since the Cedar engine can examine policy scope before evaluating the condition logic, it can pre-filter policies at runtime, considering only those relevant to the request. Figure 8.1 shows the different flows for policy evaluation in OPA and Cedar.